Ciclo de instrucciones

Federico Velásquez de Luna. Arquitectura de Computadoras. Ingeniería en Sistemas Computacionales. Universidad Autónoma de Aguascalientes

Resumen*—En este reporte se explicara como funciona una serie de operaciones conocidas como el "ciclo de instrucciones". Se detallara como funciona cada etapa de dicho ciclo: *fetch, decode, execute* y *memory/write back. Asimismo, se vera un ejercicio como ejemplo en el que se realizan operaciones simples, utilizando un arquitectura imaginaria. De modo que sea mas fácil entender el ciclo.

I. Introducción

Dentro de la arquitectura de un procesador podemos encontrar una serie de instrucciones que son fundamentales para que cualquier sistema computacional pueda funcionar. A esta serie de instrucciones se le conoce como "el ciclo de instrucciones". Cuando realizamos una acción, como lo es un algoritmo o presionar una tecla, ocurre una secuencia de acciones que permiten que se realice de manera correcta. La GPU es la encargada de administrar cada paso para que se puedan ejecutar.

Las operaciones que suceden el Ciclo de Instrucciones son la siguientes:

- Fetch: es la primera etapa del ciclo en donde se toman las instrucciones que hay en la memoria RAM asignada por el procesador a través de una serie de unidades y registros. En ese memorias es donde el GPU toma la información necesaria para comenzar a leer las instrucciones.

- Decode: en esta etapa donde se interpreta la instrucción y determina cual es la operación que se va a realizar, así como los registro y los datos necesarios.

- Execute: en esta parte del ciclo, el procesador ya sabe que operación se trata y cuales son los parámetros que necesita, para después ejecutarla de manera apropiada. Muchas instrucciones no se ejecutan de la misma manera ya que la forma de utilizar el hardware depende del objetivo de la instrucción. Tenemos instrucciones de movimientos de bits, aritméticas, de salto y a memoria.

- Memory/Write back: según los resultados obtenidos de la operación que se realizo anteriormente, se accede a memoria y los guarda. Puede ser en registros internos del procesador o de nuevo en RAM.

A continuación veremos un poco mas a detalle como funciona el Ciclo de Instrucciones. Para ello, utilizaremos una arquitectura imaginara para realizar operaciones simples.

II. Metodología

Se nos presenta lo siguiente:

1. Basandose en la arquitectura imaginaria, traducir a lenguaje máquina (binario) el algoritmo destino.
2. Una vez teniendo las instrucciones codificadas, simular el ciclo de instrucciones con cada una de sus etapas instrucción por instrucción.

Codificación de instrucción:

- Aritméticas: el resultado se guarda en la memoria 1.

b0 b1 b2 (Operación) b3 b4 b5(Operando A) b6 b7 b8 (Operando B)

Cada instrucción sera de 9 bits de longitud, la cual se va a separar en grupos de 3 y cada uno de ellos representa un apartado diferente; los primeros tres corresponden al operando (suma, resta, incremento o decremento). El segundo conjunto será el primer operando, mientras que el tercer conjunto será el segundo operando.

- Memoria:

b0 b1 b2 (Memoria origen) b3 b4 b5 (Memoria destino)

En este tipo de instrucciones solo cuentan con 6 bits de longitud y están divididos en conjuntos de 3 bits. Tienen la función de mover lo que contiene un memoria en especifico hacia otra. De manera que en nuestra instrucción, el primer conjunto pertenece a la memoria origen, mientras que el otro conjunto pertenece a la memoria destino.

La manera de interpretar las operaciones con bits es la siguiente:

1. Suma: 000
2. Resta: 001
3. Incremento (+1): 010
4. Decremento (-1): 011
5. Mover a memoria: 100

Para poner en práctica el Ciclo de Instrucciones, tenemos los siguientes algoritmos, y vamos a ir llenando una tabla con cada uno de los pasos necesarios para poder realizar cada uno de los pasos del ciclo.

Algoritmos:

1) Memoria2 = 2 + 2
2) Memoria3 = 5 + 1
3) Memoria4 = Memoria2
4) Memoria5 = 2 – 1

1) Memoria2 = 2 + 2

- Fetch:

En esta etapa se accede a memoria y se extrae la instrucción para poder trabajar con ella, para ello es necesario traducir los algoritmos a binario para poder mandarlo a la siguiente etapa.

Podemos observar en el algoritmo que se realizaran dos instrucciones: una para realizar la suma y otra para acceder a la memoria2 y escribir el resultado ahí. Primero haremos la conversión de la operación y luego la del movimiento de la memoria.

Para poder convertir las instrucción a binario vamos siguiendo el formato que vimos anteriormente, nuestro primer conjunto seria la operación que realizaremos, para este caso, es una suma. Para los otros dos conjuntos de números serian nuestros operadores: para el operador A es 2 y para el operador B es 2. Por lo tanto, tenemos:

Operación: Suma = 000
Operando A: 2 = 010 (en binario)
Operando B: 2 = 010 (en binario)

La instrucción completa seria: <u>000 010 010</u>

Ahora, para convertir la instrucción para guardar el resultado de la operación en la memoria2: también seguimos el formato que vimos anteriormente. En donde el primer conjunto de bits pertenece a operación que se realizara, en este caso, sería la de mover memoria. Para el segundo conjunto, a la memoria origen, que es la memoria1. Y el tercer conjunto de bits representa a la memoria destino, la cual es la memoria 2. Por lo que obtenemos:

Operación: mover memoria = 100
Memoria origen: memoria1 = 001
Memoria destino: memoria2 = 010

La instrucción junta sería: <u>100 001 010</u>

Ahora que tenemos las dos instrucciones traducidas, vamos llenando la tabla de cada paso del ciclo para que la instrucción se ejecute correctamente.

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria2 = 2 + 2	000 010 010			
	100 001 010			

- Decode:

En esta etapa se encarga de leer la instrucción que se recibió y desglozarla para determinar cada uno de los apartados de esta. De esta manera, se identifica cual es la operación que realizara y con que.
Para la arquitectura que se esta usando para esta práctica, se tomaria la instrucción así:

Para la operación 000 010 010:
Operación = 000 (Suma)
Operador A = 010 (2)
Operador B = 010 (2)

Por lo que obtendriamos:

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria2 = 2 + 2	000 010 010	Opreación = suma A = 2 B = 2		
	100 001 010			

- Execute:

En este punto del proceso, se recibe cual es la operación y con que se va a realizar. Por lo que solo se encarga de realizar dicha operación. En este caso, solo seria sumar 2 + 2, dando como resultado 4.

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria2 = 2 + 2	000 010 010	Opreación = suma A = 2 B = 2	2 + 2 = 4	
	100 001 010			

- Write back:

Esta es la última parte del proceso, en donde se accede a la memoria y se guarda el resultado de nuestra instrucción. Para este caso, nuestra arquitectura nos dice que el resultado se guarda en la memoria1.

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria2 = 2 + 2	000 010 010	Opreación = suma A = 2 B = 2	2 + 2 = 4	Memoria1 = 4
	100 001 010			

Ahora es turno de la segunda instrucción, la cual nos dice que se accedera a la memoria2 y se escribirá el resultado de nuestra instrucción pasada. Volveremos a seguir el proceso, pero ahora empezando desde el decode.

- Decode:

En esta parte del proceso, se tomaría la instrucción de la siguiente manera:

Operación = 100 (mover memoria)
Memoria origen = 001 (memoria 1)
Memoria destino = 010 (memoria 2)

Así obtenemos:

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria2 = 2 + 2	000 010 010	Opreación = suma A = 2 B = 2	2 + 2 = 4	Memoria1 = 4
	100 001 010	Operación = mover memoria Memoria origen = M1 Memoria destino = M2		

- Execute:

Ahora la operación se ejectua para este etapa, entonces se mueve el contenido de la memoria1 a la memoria2.

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria2 = 2 + 2	000 010 010	Opreación = suma A = 2 B = 2	2 + 2 = 4	Memoria1 = 4
	100 001 010	Operación = mover memoria Memoria origen = M1 Memoria destino = M2	Memoria 1 → Memoria 2	

- Write back:

Para finalizar, simplemente se guarda en la memoria 2 lo que tenga la memoria 1, que es el resultado de la operación pasada: 4.

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria2 = 2 + 2	000 010 010	Opreación = suma A = 2 B = 2	2 + 2 = 4	Memoria1 = 4
	100 001 010	Operación = mover memoria Memoria origen = M1 Memoria destino = M2	Memoria 1 → Memoria 2	Memoria2 = 4

Y así terminaría el proceso para cada etapa del ciclo de instrucciones para el primero algoritomo.

2) Memoria3 = 5 + 1

- Fetch:

Para este algoritmo, primero convertimos nuestras instrucciones a número binario. En este algoritmo se trata de un incremento a un número.

Para la primera instrucción:

Operación: incremento + 1 (010)
Operando A: 5 (101)
El segundo operando no es necesario de ponerlo, ya que la misma operación sabe realizar el incremento en 1.

La instrucción completa queda: <u>010 101</u>

Para la segunda instrucción:

Operación: mover memoria (100)
Memoria origen: Memoria1 (001)
Memoria destino: Memoria3 (011)

La instrucción completa queda: <u>100 001 011</u>

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria3 = 5 + 1	010 101			
	100 001 011			

- Decode:

Ahora esta etapa se encarga de procesar cada uno de los apartados de la instruccón para poder mandarlos a la siguiente etapa y realizar la operación.

Para la instrucción: 010 101
Operación: 010 (incremento en 1)
Operando: 101 (número 5)

Así obtenemos:

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria3 = 5 + 1	010 101	Operación: incremento en 1 Operando A: 5		
	100 001 011			

- Execute:

Ahora que ya sabemos con que vamos a trabajar, realizamos el incremento en 1 para el operando 5, dando como resultado 6.

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria3 = 5 + 1	010 101	Operación: incremento en 1 Operando A: 5	5 + 1 = 6	
	100 001 011			

- Write back:

Para finalizar, se accede a la memoria1 por defecto para escribir el resultado de nuestra operación, o sea, el número 6.

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria3 = 5 + 1	010 101	Operación: incremento en 1 Operando A: 5	5 + 1 = 6	Memoria1 = 6
	100 001 011			

Ahora pasamos con la instrucción encargada de escribir el resultado de la operación en la memoria3.

- Decode:

La instrucción se estaria procesando de la siguiente manera:

Operación: mover memoria (100)
Memoria origen: memoria1 = (001)
Memoria destino: memoria3 = (011)

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria3 = 5 + 1	010 101	Operación: incremento en 1 Operando A: 5	5 + 1 = 6	Memoria1 = 6
	100 001 011	Operación: mover memoria Memoria origen = M1 Memoria destino: M3		

- Execute:

Ahora, se realiza la ejecución de la operación: se estara moviendo el contenido de la memoria1 a la memoria3.

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria3 = 5 + 1	010 101	Operación: incremento en 1 Operando A:	5 + 1 = 6	Memoria1 = 6
		5		
	100 001 011	Operación: mover memoria Memoria origen = M1 Memoria destino: M3	Memoria1 → Memoria3	

- Memory/ Write Back

Finalmente, se accede a la memoria3 y se escribe el contenido que se guardo de la operación en la memoria1, que es el valor 6.

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria3 = 5 + 1	010 101	Operación: incremento en 1 Operando A: 5	5 + 1 = 6	Memoria1 = 6
	100 001 011	Operación: mover memoria Memoria origen = M1 Memoria destino: M3	Memoria1 → Memoria3	Memoria3 = 6

Y ya esta terminado el proceso del Ciclo de Instrucciones para este algoritmo.

3) Memoria4 = memoria2

- Fetch:

Primero convertimos nuestro algoritmo en binario para que las siguientes etapas del proceso puedan trabajar de manera adecuada. En este caso solo se realiza una instrucción, porque no se realiza otra operación ademas del movimiento de memoria.

Operación: mover memoria (100)
Memoria origen: memoria2 (010)
Memoria destino: memoria4 (100)

Nuestra instrucción queda: <u>100 010 100</u>

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria4 = memoria2	100 010 100			

- Decode:

En esta etapa se recibe la instrucción y la comienza a interpretar dependiendo la arquitectura del procesadores. Para este caso, sería de la siguiente manera:

Instrucción: 100 010 100
Operación: 100 = mover memoria
Memoria origen: 010 = memoria2
Memoria destino: 100 = memoria4

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria4 = memoria2	100 010 100	Operiación: mover memoria Memoria origen: memoria2 Memoria destino: memoria4		

- Execute:

Ahora que ya sabemos cuales datos tenemos y que operación se va a realizar con ellos, se ejecuta la operiación. La cual es mover la memoria2 a la memoria4.

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria4 = memoria2	100 010 100	Operiación: mover memoria Memoria origen: memoria2 Memoria destino: memoria4	Memoria2 → Memoria4	

- Write back

Para finalizar, se accede a la memoria4 y se escribe el contenido de la memoria2, dando por concluido el proceso del Ciclo de Instrucciones para este algoritmo.

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria4 = memoria2	100 010 100	Operiación: mover memoria Memoria origen: memoria2 Memoria destino: memoria4	Memoria2 → Memoria4	Memoria4 = memoria2

4) Memoria5 = 2 - 1

- Fetch:

Como primer paso para este proceso: convertimos nuestro algoritmo en binario para que las demas partes del proceso pueden trabajar de manera adecuada. Para este caso, el algoritmo se trata de un decremento en 1.

Para la primera instrucción:

Operación: decremento - 1 (011)
Operando A: 2 (010)
El segundo operando no es necesario de ponerlo, ya que la misma operación sabe realizar el decremento en 1.

La instrucción completa queda: 011 010

Para la segunda instrucción:

Operación: mover memoria (100)
Memoria origen: Memoria1 (001)
Memoria destino: Memoria5 (101)

La instrucción completa queda: 100 001 101

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria5 = 2 - 1	011 010			
	100 001 101			

- Decode:

Ahora esta etapa se encarga de procesar cada uno de los apartados de la instruccón para poder mandarlos a la siguiente etapa y realizar la operación.

Para la instrucción: 011 010
Operación: 011 = decremento en 1
Operando: 010 = número 2

Así obtenemos:

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria5 = 2 - 1	011 010	Operación: decremento en 1 Operando A: 2		
	100 001 101			

- Execute:

Ahora que ya sabemos con que vamos a trabajar, realizamos el decremento en 1 para el operando 2, dando como resultado 1.

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria5 = 2 - 1	011 010	Operación: decremento en 1 Operando A: 2	2 - 1 = 1	
	100 001 101			

- Write back:

Para finalizar, se accede a la memoria1 por defecto para escribir el resultado de nuestra operación, o sea, el número 1.

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria5 = 2 - 1	011 010	Operación: decremento en 1 Operando A: 2	2 - 1 = 1	Memoria1 = 1
	100 001 101			

Ahora pasamos con la instrucción encargada de escribir el resultado de la operación en la memoria5.

- Decode:

La instrucción se estaria procesando de la siguiente manera:

Operación: mover memoria (100)
Memoria origen: memoria1 = (001)
Memoria destino: memoria5 = (101)

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria5 = 2 - 1	011 010	Operación: decremento en 1 Operando A: 2	2 - 1 = 1	Memoria1 = 1
	100 001 101	Operación: mover memoria Memoria origen: memoria1 Memoria destino: memoria5		

- Execute:

Ahora, se realiza la ejecución de la operación: se estara moviendo el contenido de la memoria1 a la memoria5.

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria5 = 2 - 1	011 010	Operación: decremento en 1 Operando A: 2	2 - 1 = 1	Memoria1 = 1
	100 001 101	Operación: mover memoria Memoria origen: memoria1 Memoria destino: memoria5	Memoria1 → Memoria5	

- Memory/ Write Back

Finalmente, se accede a la memoria5 y se escribe el contenido que se guardo de la operación en la memoria1, que es el valor 1.

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria5 = 2 - 1	011 010	Operación: decremento en 1 Operando A: 2	2 - 1 = 1	Memoria1 = 1
	100 001 101	Operación: mover memoria Memoria origen: memoria1 Memoria destino: memoria5	Memoria1 → Memoria5	Memoria5 = 1

Y ya esta terminado el proceso del Ciclo de Instrucciones para este algoritmo.

III. Resultados

Ahora podemos ver cada tabla para los algoritmos correspondientes, podemos como se fue realizando paso a paso cada instrucción para que al final se pueda ejecutar nuestra operación de manera adecuada.

1) Memoria2 = 2 + 2

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria2 = 2 + 2	000 010 010	Opreación = suma A = 2 B = 2	2 + 2 = 4	Memoria1 = 4
	100 001 010	Operación = mover memoria Memoria origen = M1 Memoria destino = M2	Memoria 1 → Memoria 2	Memoria2 = 4

2) Memoria3 = 5 + 1

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria3 = 5 + 1	010 101	Operación: incremento en 1 Operando A: 5	5 + 1 = 6	Memoria1 = 6
	100 001 011	Operación: mover memoria Memoria origen = M1 Memoria destino: M3	Memoria1 → Memoria3	Memoria3 = 6

3) Memoria 4 = memoria 2

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria4 = memoria2	100 010 100	Operiación: mover memoria Memoria origen: memoria2 Memoria destino:	Memoria2 → Memoria4	Memoria4 = memoria2

		memoria4		

4) Memoria5 = 2 - 1

Instrucción	Fetch	Decode	Execute	Memory/ Write Back
Memoria5 = 2 - 1	011 010	Operación: decremento en 1 Operando A: 2	2 - 1 = 1	Memoria1 = 1
	100 001 101	Operación: mover memoria Memoria origen: memoria1 Memoria destino: memoria5	Memoria1 → Memoria5	Memoria5 = 1

IV. Conlusiones

A lo largo de este reporte se puede observar la importancia que tiene la sincronización de los componentes internos y como la ejecución de cualquier programa computacional depende totalmente de ello. Mediante el uso de la arquitectura imaginaria que se utilizó para esta práctica, pudimos observar como una instrucción en binario por las diferentes etapas del Ciclo de instrucciones; como primero primero busca en la memoria las instrucciones (fetch), la interpretación de dicha instrucción (decode), la ejecución de la operación a realizar (execute) para despues guarda el resultado en la memoria (memory/write back). Todo este proceso se reduce a una ejecución metódica y sincronizada por la que pasa cualquier equipo computacional para realizar cualquier operación. Comprender como funciona esta dinámica en una arquitectura sencilla, permite mostrarnos el funcionamiento de los procesadores actuales, lo cuales realizan estas operaciones unas miles de millones de veces por segundos.

Referencias

[1] Hardzone, “Así es como tu CPU ejecuta las instrucciones que le da el software”. [Online]. Available: https://hardzone.es/tutoriales/rendimiento/ciclo-instruccion-cpu/

[2] Lenovo, “¿Qué es Decode?”. [Online]. Available: https://www.lenovo.com/gb/en/glossary/what-is-decode/?orgRef=https%253A%252F%252Fwww.google.com%252F